Gravitas policy registry review

What this package is

A local review package with a gated dummy token, a transfer manager, a registry-backed API and reproducible checks. It demonstrates what the existing registry does and what a token must add to enforce a transfer rule. The new contracts are separate from the existing registry and do not alter it.

Start the sandbox

From the repository root, run one command:

```
docker compose -f mb-sandbox-review/docker-compose.yml up --build
```

The container starts a local chain, deploys the registry, plain GUSD, the dummy gated sukuk and manager, approves the token, registers the manager, runs the tests and seven scenario checks, then starts the API. It prints addresses, policy version, a local development key and sample requests. Both ports bind to loopback. No public deployment is made.

API specification: `mb-sandbox-review/reference/api/openapi.yaml` (registry-backed reference API). The earlier file-driven mock specification describes a different service and is superseded for this review.

Three answers

Allowed or refused? The manager calls the registry and transfers the dummy gated token in one transaction. An approved asset and registered manager can proceed. An unapproved asset, unregistered manager or paused registry is refused. The registry read alone does not move tokens.

Can the transfer check be bypassed? Not through the reference gated token's direct or delegated routes: its `_update` hook asks the registry on balance-changing operations. The original plain GUSD remains transferable without a registry call. The guard applies only to `GatedDemoSukuk`; it does not change GUSD or the registry.

What if the API or registry is unavailable? The API reads live state at one pinned block. After a two-second timeout and one retry, it returns HTTP 503 with `DENY / REGISTRY_UNAVAILABLE`. A paused registry returns `DENY / REGISTRY_PAUSED`. It never serves a cached ALLOW. There is no production transfer client.

Glossary

- **Gated token** — A token that checks registry state before a balance-changing operation.
- **Executor** — A contract registered to call the registry's subscription check.
- **Approved asset** — A token whose registry asset flag is true.
- **Policy version** — A counter advanced by each recorded registry rule change.
- **Policy hash** — A commitment to a rule change and the preceding hash.
- **Pinned block** — The chain block number used for all reads in one API decision.
- **Pause** — The registry state that makes guarded checks fail.

2. Run and inspect

The Docker image provides Node 22, Foundry 1.9.0 and Solidity 0.8.24 through the separate reference Foundry configuration. Its build needs network access for the pinned binaries and source dependency. The

optional `.devcontainer` route uses the same image and requires the reviewer's own account with a cloud development environment; forwarded ports are private.
A second terminal runs the full verification and returns a pass or fail code:

```
docker compose -f mb-sandbox-review/docker-compose.yml exec sandbox \
  bash /workspace/mb-sandbox-review/reference/scripts/verify-all.sh
```

Startup prints the local registry, GUSD, gated token and manager addresses, policy version, API URL and development API key. The default mnemonic and key are public test values, not secrets. They must never be used on a public chain or reused elsewhere. Host ports 8545 and 8787 are bound to `127.0.0.1`.

Native run without Docker

On Linux, install Git, Bash, Node 22, Foundry 1.9.0 (`forge`, `cast`, `anvil`), Python 3 with `venv`, and `curl`. Clone this exact release with its recursive submodules; `forge-std` is nested inside OpenZeppelin:

```
git clone --recurse-submodules --branch mb-review-v3 \
  https://github.com/AbZe628/gravitas-protocol.git
cd gravitas-protocol
git submodule update --init --recursive
[ -e lib/forge-std ] || ln -s openzeppelin-contracts/lib/forge-std lib/forge-std
python3 -m venv .venv-review
source .venv-review/bin/activate
pip install openapi-spec-validator==0.7.2 pycryptodome==3.23.0
export ANVIL_HOST=127.0.0.1 API_HOST=127.0.0.1
bash mb-sandbox-review/reference/scripts/start-sandbox.sh
```

In a second terminal from the same checkout, activate `.venv-review`, set `ANVIL_HOST=127.0.0.1`, then run `bash mb-sandbox-review/reference/scripts/verify-all.sh`. The start script runs the local builds, contract and API suites, seven scenarios, deployment and API; the second command additionally checks a real local-node outage and recovery. These native steps need network access for package and compiler downloads. Only the linked GitHub Actions run below verifies the container build and lifecycle.
The API accepts `X-API-Key`. A local policy request is:

```
curl -H 'X-API-Key: local-reference-key-only' \
  http://127.0.0.1:8787/v1/policy
```

The Compose file, Dockerfile and startup script are versioned in the release.

3. What the reference code does

`GatedDemoSukuk` asks the registry from `_update`, covering direct transfer, delegated transfer, mint and burn after construction. The constructor's first mint is exempt: the token cannot be approved before it has an address. The tests assert that refused routes leave balances unchanged. This is why the existing registry alone cannot close the plain-token bypass: it stores and answers policy questions but it cannot intercept a token that never calls it.

`DemoSukukManager.transferSukuk` checks that the token is approved and that the manager is an authorized executor. It then calls `transferFrom` and emits `TransferChecked` with the request ID, token, parties, amount, returned policy version and hash. Revert rolls back the transfer and event. The source is GatedDemoSukuk.sol and DemoSukukManager.sol; see the reference tests.

The manager's registry call originates from the manager contract. The registry accepts a subscriber argument but does not use it to make an eligibility decision. The API also classifies asset state only. Neither component identifies an investor or checks the instrument's terms.

While paused, the current reference token freezes transfer, mint and burn. That trade-off is tested. A real issuer must choose and test its pause policy: keep a full freeze, or separately govern a narrow emergency exit. This package implements only the full freeze.

Reference API boundary

`GET /health` reports service liveness; it does not prove that the registry is reachable. `GET /v1/policy` reads the current policy version and hash, block, chain ID, registry address, timestamp and pause flag. `POST /v1/verify` accepts an asset address, a decimal amount string and request context. The response records an asset decision, reason code, policy version/hash and the block used. The amount is validated and logged; it is not an investor-eligibility or instrument-terms check.

The reader first fetches one latest block and performs the state reads against that same block tag. It does not return an old ALLOW on RPC error. All routes require the API key. Requests and outcomes are appended to local JSONL; invalid or unavailable reads do not invent a policy version, hash or block. The service and reader are server.mjs and registry.mjs.

4. Scenarios 01 and 02

01 — Dummy sukuk transfer

The local deployment creates plain GUSD, a dummy gated sukuk and the manager. The owner approves the token and registers the manager. The test checks a successful `transferFrom`, the recipient balance and the manager event's current version and `policyHistory(version)` hash.

Run:

```
bash mb-sandbox-review/scripts/run-scenario-1.sh
```

Plain GUSD has no registry-aware transfer hook. The reference token checks its own approval on each balance-moving route. This is a dummy local transfer, not a real subscription or sukuk issuance. See the scenario source.

02 — Approved asset pair

The existing registry view returns true for a pair only when both asset flags are approved. That view does not move tokens or constrain a token that ignores the registry. The runner checks both the existing view and a successful transfer through the gated token and manager.

Run:

```
bash mb-sandbox-review/scripts/run-scenario-2.sh
```

See the scenario source and runner.

What the assertions establish

Case	Setup and action	Observable result
01 — Transfer	Approved gated token, registered manager and holder allowance; request a 100-token transfer.	Holder decreases by 100; recipient increases by 100; manager event matches the current version and registry hash. See captured run 01.
02 — Pair and gate	Query two registry-approved assets, then transfer the approved token through the registered manager.	Pair view returns true for the approved pair; transfer succeeds. The revoked-asset refusal is checked in Scenario 03. See captured run 02.

5. Scenarios 03 and 04

03 — Unapproved asset or executor

An unapproved asset fails with the registry's exact asset-compliance revert. A caller that is not registered as an executor fails with the exact caller-authorization revert. Both paths preserve balances. The registry checks the manager as caller; it does not check subscriber identity or eligibility.
Run:

```
bash mb-sandbox-review/scripts/run-scenario-3.sh
```

See the scenario source.

04 — Rule change and policy history

Revoking the token advances the policy version and changes the hash. The earlier version's hash is unchanged. The new commitment is recomputed from the previous hash, `asset`, token address and `false`. Re-approval advances the version again; it does not roll back history.
Run:

```
bash mb-sandbox-review/scripts/run-scenario-4.sh
```

The manager event refers to the registry version/hash used in the transfer transaction. The commitment identifies an ordered policy change; it does not contain a ruling or prove an instrument's status. See the scenario source.

Expected checks

Change	Expected check
Asset revoked	Exact revert `GPR: Asset not Shariah-compliant`; holder and recipient balances do not change.
Manager not registered	Exact revert `GPR: Calling contract not an authorized executor`; no token movement.
Version N revoked	Version advances to N+1; prior hash remains; new hash uses `keccak256(abi.encode(previousHash, keccak256("asset"), token, false))`.
Asset re-approved	Version advances again; transfer can proceed with the new current version and hash.

The captured outputs are Scenario 03 and Scenario 04.

6. Scenarios 05 and 06

05 — Pause or unavailable node

A pause blocks the manager and direct gated-token transfer. The live API returns `DENY / REGISTRY_PAUSED`. If Anvil stops, the API returns HTTP 503 with `DENY / REGISTRY_UNAVAILABLE` after its configured timeout and retry. When the same local chain state is restored, a fresh live read can return ALLOW.

Run the contract checks:

```
bash mb-sandbox-review/scripts/run-scenario-5.sh
```

The full verifier also pauses the actual local registry, checks the API response, stops the actual local chain, checks 503, restores the saved state and checks recovery. This behavior is in the outage and pause scripts.

06 — Unauthorized changes and transfer routes

Only the registry owner can change the asset, router and executor lists. Reference direct and delegated transfers fail while the gated token is unapproved or the registry is paused. The plain GUSD contrast still transfers, since its code never asks the registry.

Run:

```
bash mb-sandbox-review/scripts/run-scenario-6.sh
```

The package does not change the old token or registry. A real issuer would still have to choose the token code, protect approval and pause authority, and test the complete issuance and redemption routes. See the scenario source.

Route comparison

State	Gated demo token	Plain GUSD contrast
Asset not approved	Direct transfer and delegated `transferFrom` both revert; balances remain unchanged.	Transfer still succeeds because this token does not call the registry.
Registry paused	Direct and delegated balance changes fail closed with the pause error.	Its transfer is still independent of the paused registry.
Registry restored	The owner unpauses; the approved, registered manager route can proceed again.	No registry recovery is needed for a plain-token transfer.

The captured outputs are Scenario 05 and Scenario 06.

7. Scenario 07 — One-transaction audit record

The manager transfer emits an ERC-20 `Transfer` and a `TransferChecked` event in the same transaction. The included local record was generated from a real local Anvil run and checked against its receipt and registry history.

Field	Recorded value
Chain and registry	Local chain 31337; `0x5FbDB2315678afecb367f032d93F642f64180aa3`
Token and manager	Token `0x9fe46736679d2d9a65f0992f2272de9f3c7fa6e0`; manager `0xcf7ed3acca5a467e9e704c703e8d87f634fb0fc9`
Sender	`0x70997970C51812dc3A010C7d01b50e0d17dc79C8`
Recipient	`0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC`
Amount	One token with 18 decimals
Transaction	`0xd45c98188300d453bcb1a4f412098e3bc013a350a33dc66e3230fbb74d71283f`
Block and timestamp	Block 11; `2026-11-09T20:16:54Z`
Policy	Version 3; `0x96405a9315fa1a542184d8419512d74374ec2fff1684eaca112f6b36a723033f`

The verifier decodes both logs, checks their transaction and block, reads `getPolicyVersion` and `policyHistory` at the receipt block, replays the registry changes in order and recomputes every hash. The verifier source and captured output are included. Run `bash mb-sandbox-review/scripts/run-scenario-7.sh` to produce and verify a new local record.

The older composed example combines separate operations; it is not evidence of an executed transfer. Neither example identifies an investor or establishes the legal or religious status of an instrument.

8. What production work remains

Area	Status
Transfer enforcement	**Reference implementation exists (unaudited).** Production work remaining: issuer-specific token behavior, issuer permissions, real issuance and redemption integration, and independent review.
One-transaction record	**Reference implementation exists (unaudited).** Production work remaining: approved record schema, storage and retention, monitoring, reconciliation, identity binding and operational controls.
Registry-backed HTTP and outages	**Reference implementation exists (unaudited).** Production work remaining: key management, infrastructure and network hardening, monitoring, privacy controls and an approved deployment.
Investor eligibility	**Not built.** The registry ignores the subscriber argument; no identity or investor eligibility check exists.
Ruling link	**Not built.** No ruling identifier, source document or approval workflow is connected to registry changes.
Governance changes	**Not built.** The owner-controlled registry has no new timelock, governance flow or separately managed emergency role in this package.
Independent audit	**Not built.** The reference contracts and API have not received an independent audit.

Area	Status
Funded shared testnet	**Not built.** No shared testnet or funded account was created.
Hosted API with managed keys	**Not built.** The API runs locally and prints a development key.

`checkSubscriptionCompliance(subscriber, token)` accepts a subscriber address but the registry does not inspect it. The API checks asset approval and policy state only. These components do not screen investors.

9. Candidate sequence before any production use

Every stage below is **not built**. It is a decision and work sequence, not a deployment commitment.

Stage	Required work
1. Define the instrument and policy	Obtain issuer-approved asset and ruling identifiers, document the rule set and decision authority, and define which transfer, redemption, mint and burn events are in scope. Not built.
2. Set eligibility and governance	Decide how investor identity and eligibility are established; choose the approval, pause, revocation and emergency-exit authorities; define review, delay, separation of duties and evidence retention. Not built.
3. Integrate issuer systems	Adapt and test token, manager, subscription and redemption flows against the chosen instrument. Define stable request IDs, how policy versions are bound to transactions and how rejected operations are reconciled. Not built.
4. Operate the API and records	Select hosting, key custody, access control, privacy and retention rules, monitoring, alerting, backups and recovery tests. Define what happens if the registry, RPC, API or audit sink is unavailable. Not built.
5. Independent security review	Complete a threat model, dependency review, independent contract and service audit, remediate findings, and repeat tests on the actual deployment configuration. Not built.
6. Staged deployment and approval	First obtain issuer sign-off for a public test environment and funding, then validate operations and monitoring. Mainnet deployment requires separate approvals, verified addresses, key ceremonies, incident procedures and a release decision. None of this has been done.

The sequence must be revised for the issuer's legal, operational and instrument requirements. The local prototype only informs the engineering questions; it does not satisfy any stage above.

10. Verification results

Local results

Check	Result
Existing root Foundry project	Build passed; 86 tests passed, 0 failed, 0 skipped across 9 suites.
Registry bytecode	Before/after compiled runtime hashes match; detailed comparison is in the dated evidence.
Reference Foundry project	8 passed, 0 failed, 0 skipped; 256 fuzz runs.

Check	Result
Scenario scripts	All seven runner scripts were run. Their focused outputs are captured in the dated evidence directory.
Reference API tests	9 passed, 0 failed. Includes auth, input checks, paused registry, RPC outage, timeout/retry and recovery.
Live API	Registered token ALLOW, unknown token DENY, invalid API key 401; actual pause DENY and recovery ALLOW.
Actual node outage	Local Anvil was stopped; API returned HTTP 503 and `REGISTRY_UNAVAILABLE`; saved chain state was restored and a new ALLOW followed.
Manager record	Actual receipt and paired events verified; ordered policy hash chain recomputed.
API specification and dependencies	OpenAPI 3.0.3 validation passed; API package audit reported zero vulnerabilities.

The matching runtime SHA-256 captured before and after is:

```
b058d10c887059aafd4e2ec309098ecaece7057f93a33b31b4f5ff6868f1d92f
```

Test scope

The 86 root tests cover the existing project. The separate reference suite covers the gated token and manager; 256 fuzz runs target refused transfers. The seven runner scripts were executed against the private local chain. GitHub Actions separately verified the complete Docker path; see the successful run below.
The bytecode comparison covers only the registry runtime. It does not establish security of the reference token, manager, API, dependencies or deployment setup. The API and transfer tests prove the specific checked outcomes in this local configuration.

11. Limits and outstanding checks

Hosted Docker verification passed. GitHub Actions run 42697229864 on commit `770ee815a794281817e8defac7f268708f042c3b` (tag `mb-review-v2`) built and started the container, completed verification, saved logs and shut down successfully in about two minutes. Docker was not available locally; local component checks were run directly. The full container lifecycle is verified only by that hosted run. See the verification record and dated evidence.

Limits of this review

The reference code is unaudited and is not an issuer product. The dummy sukuk has no legal or religious standing. Tests prove only the selected behavior in the local setup. This work does not establish investor eligibility, a ruling, regulator approval, a hosted service or mainnet readiness. No public deployment was made. The full-freeze pause policy is a design choice, not a universal recommendation.
The earlier registry source review describes the original token bypass and registry limits. It should be read with the new reference implementation status above, not as a claim that the old token was changed.
For package entry points, see the README, sandbox access guide, reference API specification, Scenario 07, and verification metadata.